

COM 212

System Programming

Course Outline

1. Concepts of System Programming
2. Basic Elements of and Assembly Program
3. Assembler Functions
4. Interpretation, Translation, Compilation
5. Front End and Back End Compilation
6. Error Checking, Utilities, Libraries
7. Operating Systems: Importance, Uses, Types & Services
8. I/O Buffering, Files & Interrupts
9. Multiprogramming, Multitasking, Multiprocessing

CONCEPTS OF SYSTEM PROGRAMMING

Introduction

System programming (or systems programming) is the activity of programming system software. The primary distinguishing characteristic of systems programming when compared to application programming is that:

Application programming aims at producing software which provides services to the user (e.g. word processor, Spreadsheets, databases, accounting packages).

Systems programming aims at producing software which provides services to the computer hardware (e.g. disk defragmenter, Operating Systems...). It also requires a greater degree of hardware awareness that is to say it is machine dependent and the programmer needs to know the hardware within which the software will operate

In system programming more specifically:

- The programmer will make assumptions about the hardware and other properties of the system that the program runs on, and will often exploit those properties (for example by using an algorithm that is known to be efficient when used with specific hardware)
- Usually a low-level programming language or programming language dialect is used and does the following:
 - Operate in resource-constrained environments
 - Is very efficient and has little runtime overhead
 - Has a small runtime library, or none at all
 - Allows for direct and "raw" control over memory access and control flow
- Debugging can be difficult if it is not possible to run the program in a debugger due to resource constraints. Running the program in a simulated environment can be used to reduce this problem.

Systems programming is sufficiently different from application programming that programmers tend to specialize in one or the other.

In system programming, often limited programming facilities are available. The use of automatic garbage collection is not common and debugging is sometimes hard to do. The runtime library, if available at all, is usually far less powerful, and does less error checking. Because of those limitations, monitoring and logging are often used; operating systems may have extremely elaborate logging subsystems. Implementing certain parts in operating systems and networking requires systems programming

What is Assembler?

An assembler is a translator that translates source instructions (in symbolic language) into target instructions (in machine language), on a one-to-one basis. This means that each source instruction is translated into exactly one target instruction.

An assembler is also defined as a translator that translates a machine-oriented language into machine language. This definition distinguishes between assemblers and compilers. Compilers are translators of problem-oriented languages or of machine-independent languages.

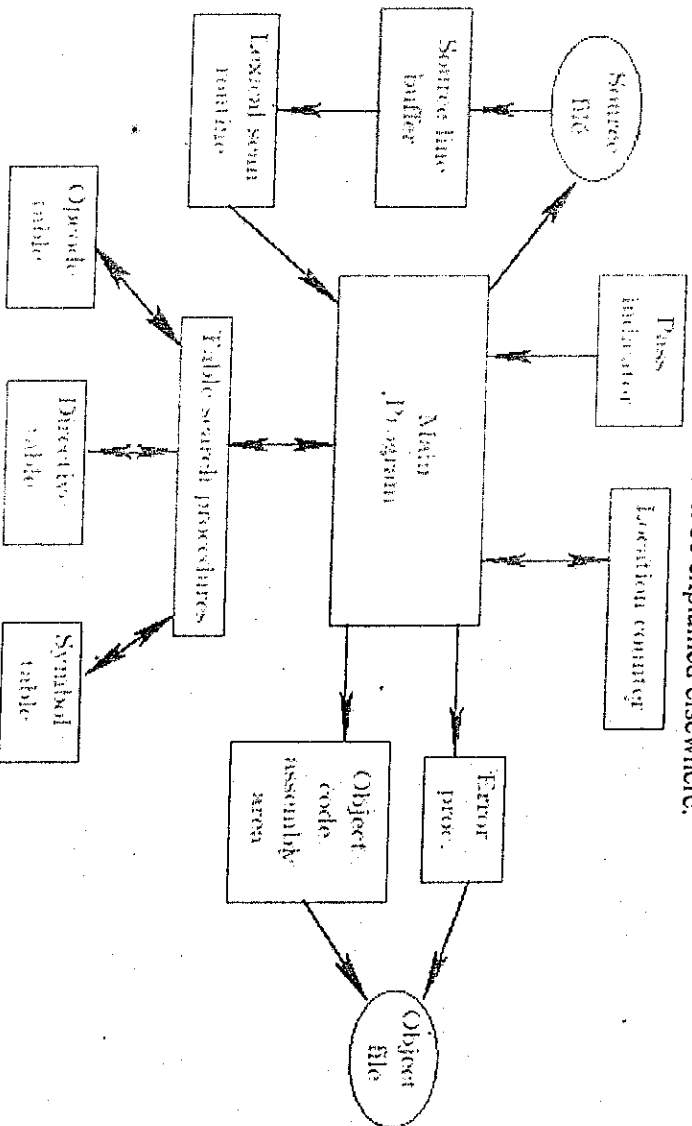
One reason for studying assemblers is that the operation of an assembler reflects the architecture of the computer. The assembler language depends heavily on the internal organization of the computer. Architectural features such as memory word size, number formats, internal character codes, index registers, and general purpose registers, affect the way assembler instructions are written and the way the assembler handles instructions and directives. This fact explains why there is an interest in assemblers today and why a course on assembler language is still required for many, perhaps even most, computer science degrees.

Assemblers are translators and they work on one program at a time. The tasks of locating, loading, and linking (as well as many other tasks) are performed by a loader. A modern assembler has two inputs and

two outputs. The first input is short, typically a single line typed at a keyboard. It activates the assembler and specifies the name of a source file (the file containing the source code to be assembled). It may contain other information that the assembler should have before it starts. This includes commands and specifications such as:

- The names of the object file and listing file. Display (or do not display) the listing on the screen while it is being generated.
- Display all error messages but do not stop for any error.
- Save the listing file and do not print it (see figure below).

This program does not use macros. The symbol table is larger (or smaller) than usual and needs a certain amount of memory. All these terms will be explained elsewhere.



Main Components and Operations of an Assembler

Types of Assemblers and Loaders

1. A One-pass Assembler: One that performs all its functions by reading the source file once.
2. A Two-Pass Assembler: One that reads the source file twice.
3. A Resident Assembler: One that is permanently loaded in memory.
4. Typically such an assembler resides in ROM, is very simple (supports only a few directives and no macros), and is a one-pass assembler.
5. A Macro-Assembler: One that supports macros
6. A Cross-Assembler: An assembler that runs on one computer and assembles programs for another. Many cross-assemblers are written in a higher-level language to make them portable. They run on a large machine and produce object code for a small machine.
7. A Meta-Assembler: One that can handle many different instruction sets.
8. A Disassembler: This, in a sense, is the opposite of an assembler. It translates machine code into a source program in assembler language.

What is a Program?

In operating systems, when a program is running, it is called a process. The program itself can be in many forms, i.e. machine code, assembly code, C code, C++ or FORTRAN, Java, etc.

Processors execute machine code ONLY. Machine code is written with binary machine words composed of opcodes and operands. The opcodes represent different instructions, for instance an arithmetic addition. Operands are operated on by the instruction. A machine word may look like:

100010001101010010101101101011

In this case, 10001000 encodes the opcode, 110101001010 and 101101101011 are the two operands. Most people don't want to write code in this format. Naturally, our fore-runners sought a way out, using mnemonics to represent all those binary streams. Then the line above may become:

ADD AX, BX

These mnemonics and some other directive commands are collectively called assembly language. The assembly program now looks more readable and maintainable:

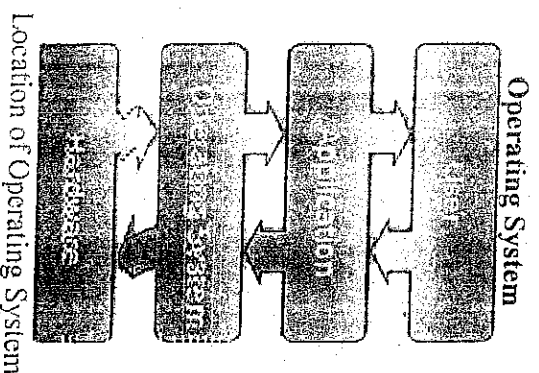
```
0000000000000100  MOV BX, VALUE1
0000000000001000  MOV AX, FACTOR
0000000000001100  MUL AX, BX
```

This is how CPU goes about to scale VALUE1 by FACTOR. VALUE1 x FACTOR doesn't make any sense to the processor. 04, 08 and 0C, written in HEX numbers, are memory addresses where corresponding lines in the program are stored.

Things became easier, but not easy enough and one could write a lot of programs using these commands.

Operating System

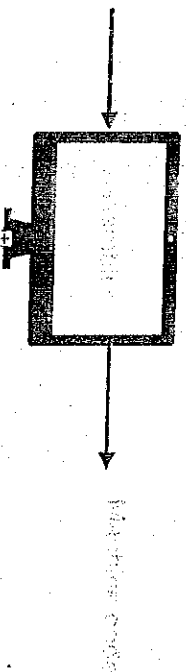
We have seen that a typically modern **assembler** creates object code by translating assembly instruction mnemonics into op-codes, and by resolving symbolic names for memory locations and other entities. The use of symbolic references is a key feature of assemblers, saving tedious calculations and manual address updates after program modifications. Most assemblers also include macro facilities for performing textual substitution---e.g., to generate common short sequences of instructions to run inline, instead of in a subroutine.



An **operating system** (commonly abbreviated *OS* and *O/S*) is the software component of a computer system that is responsible for the management and coordination of activities and the sharing of the limited resources of the computer. The operating system acts as a host for applications that are run on the machine. As a host, one of the purposes of an operating system, is to handle the details of the operation of the hardware.

Almost all computers, including handheld computers, desktop computers, supercomputers, and even video game consoles, use an operating system of some type. Some of the oldest models may however use an embedded operating system that may be contained on a compact disk or other data storage device. Common contemporary operating systems include Microsoft Windows, Mac OS, Linux, and Solaris. Microsoft Windows has a significant majority of market share in the desktop and notebook computer markets, while servers generally run on Linux or other Unix-like systems. Embedded device markets are split amongst several operating systems.

Assembler is a program for converting instructions written in low-level assembly code into relocatable machine code and generating along information for the loader.



It generates instructions by evaluating the mnemonics (symbols) in operation field and find the value of symbol and literals to produce machine code. Now, if assembler do all this work in one scan then it is called single pass assembler, otherwise if it does in multiple scans then called multiple pass assembler. Here assembler divide these tasks in two passes:

Pass-1:

- Define symbols and literals and remember them in symbol table and literal table respectively.
- Keep track of location counter
- Process pseudo-operations

Pass-2:

- Generate object code by converting symbolic op-code into respective numeric op-code
 - Generate data for literals and look for values of symbols
- Firstly, we will take a small assembly language program to understand the working in their respective passes. Assembly language statement format: [Label] [Opcode] [operand]

Example: M ADD R1, =3

where, M - Label; ADD -symbolic opcode; R1 - symbolic register.
operand; (=3) - Literal

Assembly Program:

Label	Op-code	operand	LC value (Location counter)
JOHN	START	200	
	MOVER	R1, =3	200
	MOVER	R1, X	201
L1	MOVER	R2, =2	202
	LTORG		203
X	DS	1	204
	END		205

Let's take a look on how this program is working:

1. **START:** This instruction starts the execution of program from location 200 and label with START provides name for the program. (JOHN is name for program)
2. **MOVER:** It moves the content of literal (=3) into register operand R1.
3. **MOVER:** It moves the content of register into memory operand(X).
4. **MOVER:** It again moves the content of literal (=2) into register operand R2 and its label is specified as L1.
5. **LTORG:** It assigns address to literals (current LC value).
6. **DS (Data Space):** It assigns a data space of 1 to Symbol X.
7. **END:** It finishes the program execution.

Pass-1: Define Symbol and literal table with their addresses.
 Note: Literal address is specified by LTORG or END.
Step-1: START 200 (here no symbol or literal is found so both table would be empty)

Step-2: MOVER R1, =3' 200 (=3' is a literal so literal table is made)

Literal	Address
=3'	---

Step-3: MOVEM R1, X 201
 X is a symbol referred prior to its declaration so it is stored in symbol table with blank address field.

Symbol	Address
X	---

Step-4: L1 MOVER R2, =2' 202
 L1 is a label and =2' is a literal so store them in respective tables

Symbol	Address
X	---

L1	202

Literal	Address
=3'	---
=2'	---

Step-5: LTORG 203
 Assign address to first literal specified by LC value, i.e., 203

Literal	Address
=3'	203
=2'	---

Step-6: X DS 1 204
 It is a data declaration statement i.e X is assigned data space of 1. But X is a symbol which was referred earlier in step 3 and defined in step 6. This condition is called Forward Reference Problem where variable is referred prior to its declaration and can be solved by back-patching. So now assembler will assign X the address specified by LC value of current step.

Symbol	Address
X	204
L1	202

Step-7: END 205
 Program finishes execution and remaining literal will get address specified by LC value of END instruction. Here is the complete symbol and literal table made by pass 1 of assembler.

Symbol	Address
X	204
L1	202

Literal	Address
=3'	203
=2'	205

Now tables generated by pass 1 along with their LC value will go to pass-2 of assembler for further processing of pseudo-opcodes and machine op-codes.

Pass-2:

Pass-2 of assembler generates machine code by converting symbolic machine-opcodes into their respective bit configuration (machine understandable form). It stores all machine-opcodes in MOT table (op-code table) with symbolic code, their length and their bit configuration. It will also process pseudo-ops and will store them in POT table (pseudo-op table).

Various Data bases required by pass-2:

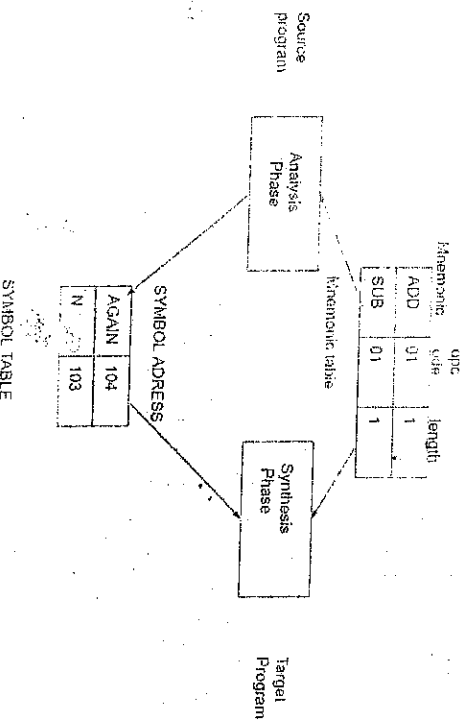
1. MOT table (machine opcode table)
2. POT table (pseudo opcode table)
3. Base table (storing value of base register)
4. LC (location counter)

Phases of an Assembler

The Assembler is a program that converts assembly language into machine language that can be executed by a computer. The Assembler operates in two main phases: Analysis Phase and Synthesis Phase. The Analysis Phase validates the syntax of the code, checks for errors, and creates a symbol table. The Synthesis Phase converts the assembly language instructions into machine code, using the information from the Analysis Phase. These two phases work together to produce the final machine code that can be executed by the computer. The combination of these two phases makes the Assembler an essential tool for transforming assembly language into machine code, ensuring high-quality and error-free software.

Analysis Phase

- The primary function performed by the analysis phase is the building of the symbol table. It must determine the memory address with which each symbolic name used in a program.
- Memory allocation is performed by using a data structure called location counter (LC).
- The analysis phase ensures that the location counter always contains the address that the next memory word in the target program should have
- At the start of its processing, it initializes the location counter to the constant specified in the START statement.
- While processing a statement, it checks whether the statement has a label.
- If so, it enters the label and the address contained in the location counter in a new entry of the symbol table
- The amount of memory needed for each assembly statement depends on the mnemonic of an assembly statement. It obtains this information from the length field in the mnemonics table.
- It obtains this information from the length field in the mnemonics table. For DC and DS statements, the memory requirement further depends on the constant appearing in the operand field, so the analysis phase should determine it appropriately.
- We use the notation <LC> for the address contained in the location counter.
- The Symbol table is constructed during analysis and used during synthesis.



The Mnemonics table is a fixed table that is merely accessed by the analysis and synthesis phases (see the arrows depicting data access). The tasks performed by the analysis and synthesis phases can be summarized as follows:

Analysis phase:

- Separate contents of the label, mnemonic opcode and operand fields of a statement.
- If a symbol is present in the label field, enter the pair (symbol, <LC>) in a new entry of the symbol table.
- Check validity of the mnemonic opcode through a look-up in the Mnemonics table.
- Perform LC processing, i.e., update the address contained in the location counter by considering the opcode and operands of the statement.

Synthesis phase:

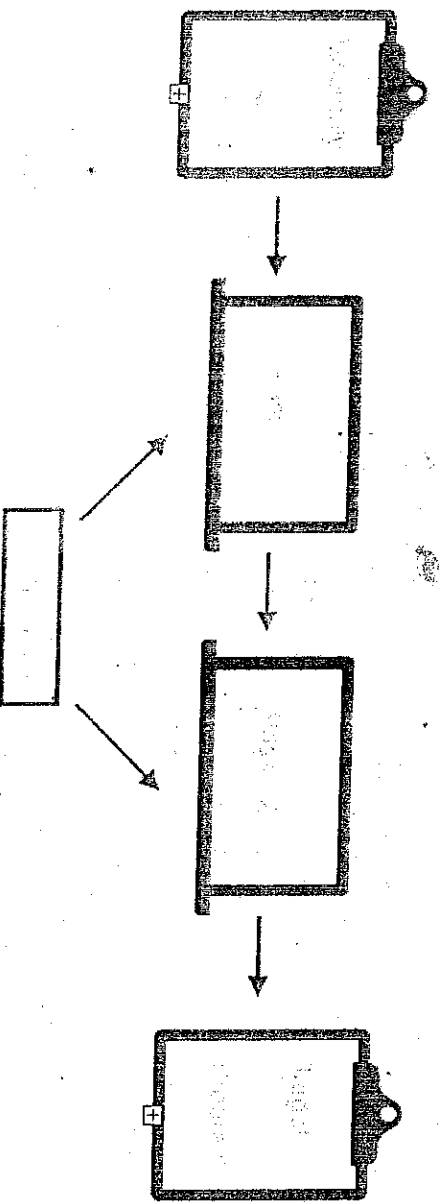
- Obtain the machine opcode corresponding to the mnemonic from the Mnemonics table.
- Obtain the address of each memory operand from the Symbol table.
- Synthesize a machine instruction or the correct representation of a constant, as the case may be.

Significance of Both Phases in Assembler

The Analysis Phase and Synthesis Phase are both essential components of the Assembler. The Analysis Phase breaks down the assembly language code into its constituent parts, such as symbols, labels, and instructions. This phase is responsible for validating the syntax of the code, checking for errors, and creating a symbol table that maps symbols to memory addresses. This phase is crucial for ensuring the code is error-free and well-formed, which helps to prevent issues during the execution of the program.

The Synthesis Phase is responsible for converting the assembly language instructions into machine code that can be executed by the computer. This phase uses the information from the Analysis Phase, such as the symbol table, to create the machine code and generate a listing file that shows the machine code and memory addresses for each instruction. The Synthesis Phase is critical for producing the final machine code that can be executed by the computer, making it the final step in the process of transforming assembly language code into machine code.

In conclusion, both the Analysis Phase and Synthesis Phase play a vital role in the Assembler. The Analysis Phase ensures the code is error-free and well-formed, while the Synthesis Phase produces the final machine code that can be executed by the computer. The combination of these two phases makes the Assembler an essential tool for converting assembly language code into machine code, which is critical for producing high-quality and error-free software.



BASIC ELEMENTS OF AN ASSEMBLY PROGRAM

Processor Registers: The registers stores data elements for processing without having to access the memory. A limited number of registers are built into the processor chip. Registers are actually available in several sizes:

- **rax** is the 64-bit, "long" size register. It was added in 2003 during the transition to 64-bit processors.
- **eax** is the 32-bit, "int" size register. It was added in 1985 during the transition to 32-bit processors.
- **ax** is the 16-bit, "short" size register. It was added in 1979, but is used in DOS or BIOS code.
- **al** and **ah** are the 8-bit, "char" size parts of the register. **al** is the low 8 bits, **ah** is the high 8 bits. They're original back to 1972.

The registers are grouped into three categories:

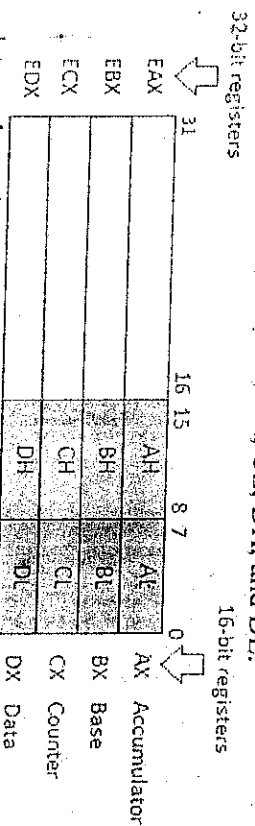
- General registers
- Control registers
- Segment registers

General Registers: The general registers are further divided into the following groups:

- Data registers
- Pointer registers
- Index registers

Data Registers: Four 32-bit data registers are used for arithmetic, logical and other operations. These 32-bit registers can be used in three ways:

- As complete 32-bit data registers: **EAX**, **EBX**, **ECX**, **EDX**.
- Lower halves of the 32-bit registers can be used as four 16-bit data registers: **AX**, **BX**, **CX** and **DX**.
- Lower and higher halves of the above-mentioned four 16-bit registers can be used as eight 8-bit data registers: **AH**, **AL**, **BH**, **BL**, **CH**, **CL**, **DH**, and **DL**.



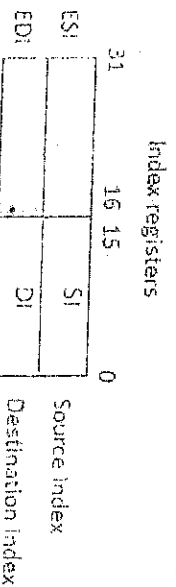
Some of these data registers has specific used in arithmetical operations.

- **AX** is the **primary accumulator**; it is used in input/output, arithmetic instructions, string load & store.
- **BX** is known as the **base register** as it could be used in indexed addressing.
- **CX** is known as the **count register** store the loop count in iterative operations.
- **DX** is known as the **data register**. It is also used in input/output operations. It is also used with **AX** register along with **DX** for multiply and divides operations involving large values.

Pointer Registers: There are three categories of pointer registers: Instruction Pointer (**IP**), Stack Pointer (**SP**) & Base Pointer (**BP**)

Index Registers: Are used for indexed addressing and sometimes used in addition and subtraction. There are two sets of index pointers:

- **Source Index (SI)** - it is used as source index for string operations
- **Destination Index (DI)** - it is used as destination index for string operations.



Control Registers: The 32-bit instruction pointer register and 32-bit flags register combined are considered as the control registers. Many instructions involve comparisons and mathematical calculations and change the status of the flags and some other conditional instructions test the value of these status flags to take the control flow to other location.

The common flag bits are:

- **Overflow Flag (OF):** indicates the overflow of a high-order bit (leftmost bit) of data after a signed arithmetic operation.
- **Direction Flag (DF):** determines left or right direction for moving or comparing string data. When the DF value is 0, the string operation takes left-to-right direction and when the value is set to 1, the string operation takes right-to-left direction.
- **Interrupt Flag (IF):** determines whether the external interrupts like, keyboard entry etc. are to be ignored or processed. It disables the external interrupt when the value is 0 and enables interrupts when set to 1.
- **Trap Flag (TF):** allows setting the operation of the processor in single-step mode. The DEBUG program we used sets the trap flag, so we could step through the execution one instruction at a time.
- **Sign Flag (SF):** shows the sign of the result of an arithmetic operation. This flag is set according to the sign of a data item following the arithmetic operation. The sign is indicated by the high-order of leftmost bit. A positive result clears the value of SF to 0 and negative result sets it to 1.
- **Zero Flag (ZF):** indicates the result of an arithmetic or comparison operation. A nonzero result clears the zero flag to 0, and a zero result sets it to 1.
- **Auxiliary Carry Flag (AF):** contains the carry from bit 3 to bit 4 following an arithmetic operation; used for specialized arithmetic. The AF is set when a 1-byte arithmetic operation causes a carry from bit 3 into bit 4.
- **Parity Flag (PF):** indicates the total number of 1-bits in the result obtained from an arithmetic operation. An even number of 1-bits clears the parity flag to 0 and an odd number of 1-bits set the parity flag to 1.
- **Carry Flag (CF):** contains the carry of 0 or 1 from a high-order bit (leftmost) after an arithmetic operation. It also stores the contents of last bit of a *shift* or *rotates* operation.

The following table indicates the position of flag bits in the 16-bit Flags register:

Flag:					O	D	I	T	S	Z		A		P		C
Bit no:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Segment Registers: The segment registers stores the starting addresses of a segment. To get the exact location of data or instruction within a segment, an offset value (or displacement) is required. Segments are specific areas defined in a program for containing data, code and stack. There are three main segments:

- **Code Segment (CS):** it contains all the instructions to be executed. A 16 - bit Code Segment register or CS register stores the starting address of the code segment.
- **Data Segment (DS):** it contains data, constants and work areas. A 16 - bit Data Segment register of DS register stores the starting address of the data segment.
- **Stack Segment (SS):** it contains data and return addresses of procedures or subroutines. It is implemented as a 'stack' data structure. The Stack Segment register or SS register stores the starting address of the stack.

Apart from the DS, CS and SS registers, there are other extra segment registers - ES (extra segment), FS and GS, which provides additional segments for storing data.

Instructions

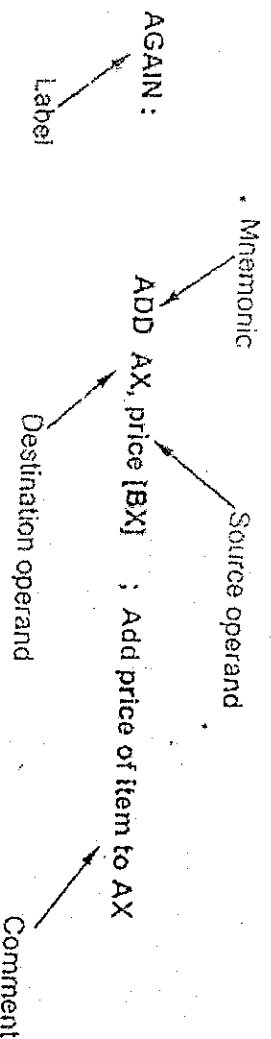
An assembly language program is a series of statements, which are either assembly language instructions such as ADD and MOV, or statements called **directives**.

An instruction tells the CPU what to do, while a **directive** (also called **pseudo-instructions**) gives instruction to the assembler. For example, ADD and MOV instructions are commands which the CPU

runs, while ORG and END are assembler directives. The assembler places the opcode to the memory location 0 when the ORG directive is used, while END indicates to the end of the source code. A program language instruction consists of the following four fields –
 [Label:] mnemonics [operands] [; comment]

A square bracket ([]) indicates that the field is optional.

Position of label, operation codes & operands



The MOV Instruction: The MOV instruction is used for moving data from one storage space to another. The MOV instruction takes two operands. The syntax of the MOV instruction is: MOV AL, 34h

Label: Label is an optional first field of an assembly statement. Labels are alphanumeric names used to define the starting location of a block of statements. When creating the executable file the assembler will replace the label with the assigned value. Labels must be unique in the executable file because an identical label encountered by the Assembler will generate an error.

Opcode (Mnemonics): Opcode is the second field in assembly language instruction. Assembly language consists of mnemonics, each corresponding to a machine instruction. Using a mnemonic you can decide what operation you want to perform on the operands. Assembler must translate each mnemonic opcode into their binary equivalent. In assembly language mnemonic form an opcode is a command such as MOV or ADD or JMP.

Operands: Next to the opcode is the operand field which might contain different number of operands. The number of operands in an instruction depends on the type of instruction, and the syntax format of the operand can also be different. Normally, the first operand is the destination of the operation. For example

MOV AL, 34h

The opcode is the MOV instruction. The other parts are called the 'operands'.

Opcodes vs operands

Each assembly language statement is split into an opcode and an operand. The opcode is the instruction that is executed by the CPU and the operand is the data or memory location used to execute that instruction.

Assembly language opcodes mnemonics and instructions	Meaning/use
INP (Input)	Inputs a value, then stores the value in the accumulator
OUT (Output)	Outputs the accumulator contents
STA (Store)	Transfers a number from the accumulator to RAM
LDA (Load)	Transfers a number from RAM to the accumulator
ADD (Add)	Adds accumulator contents to the contents at a RAM address
SUB (Subtract)	Subtracts accumulator contents from the contents at a RAM address
BRA (Branch)	When looping, jumps to the RAM memory address
HLT (Halt/Stop/End)	Stops the processor
DAT (Data definition)	Variable definition
JMP (Jump)	Jump to designated RAM address.
COMP (Compare)	Compare numbers

MOV (Move)	Used for moving data from one storage space to another
------------	--

An operand (written using hexadecimal notation) provides the data itself, or the location where the data to be processed is stored. Some instructions do not require an operand and some may require more than one operand.

Comments: Comments are messages intended only for human consumption. They have no effect on the translation process and indeed are not acted on by the Assembler. The comment field of an assembly language instruction is also optional. A semicolon signifies that the rest of the line is a comment and is to be ignored by the assembler. Following are some examples of typical assembly language statements with comments:

```

INC COUNT           ; Increment the memory variable COUNT
MOV TOTAL, 48       ; Transfer the value 48 in the TOTAL

```

ASSEMBLER FUNCTIONS

Two important steps are involved in designing software: dividing the software into smaller, more manageable components or modules, and determining how the modules will communicate. For both of these steps, the designer needs to keep the major kinds of software functionality in mind. The following are the primary functions of an assembler:

- *Generate machine code output:* This is the primary purpose of an assembler. The assembler usually generates a file copy of data and machine instructions that will later be loaded into a computer in preparation for execution. The file copy must also contain a starting address - the address of the first instruction to be executed.
- *Provide program error information for the programmer:* Assembly language programming is difficult. Assembly language programmers make mistakes. Some of these mistakes can be caught by the assembler. The ease of assembly language programming is dependent to a large extent on the quality of assembler error messages.
- *Provide machine code information for the programmer:* The assembler cannot catch all programmer errors. Some can only be detected by executing the assembled program. Then assembler can, however, provide information about the machine code that aids the programmer in debugging runtime errors. For difficult debugging problems, the programmer may need to know what code was generated and where data and instructions are located in memory.
- Assign memory for data and instructions: Early assemblers force programmers to assign memory addresses for all data and keep track of addresses assigned for instructions. Modern assembler allow programmers to use symbols (usually statement labels) to represent addresses for data or branch or jump targets. This makes the programmer's life much simpler. However, addresses are required for machine code generation. Thus the assembler must pick up the responsibility of assigning addresses to program symbols. These assignments must be remembered for use when the symbols appear in instruction operands.

DEALING WITH CHARACTERS

The input to an assembler consists of a stream of characters which represent assembly information in several different ways: integers, real numbers, and labels, quoted characters and strings, register names, and various kinds of punctuation. If character processing is mixed in with algorithms for building symbol tables or algorithms for code generation, the result is a complex mess that would challenge even the best programmers. The problem is that this organization (or disorganization) forces programmers to deal with two distinct kinds of abstraction simultaneously. The result is difficult algorithm development, a large number of errors, and difficulty in debugging. These problems are magnified if the assembler requires maintenance at a later time.

A good general design principle is to assign responsibility for different kinds of abstractions to different program modules. This principle is crucial for large designs, but is a good practice for smaller designs as well. Dividing responsibilities for different kinds of abstractions into different modules allows programmers to focus their attention on one aspect of the problem at a time.

For an assembler, the implication is that there should be a module dedicated to handling text at the level of characters. This module is called a lexical analyzer or scanner.

Assembler Modules

Thus an assembler has four main modules.

- A scanner,
- A pass 1 module,
- A pass 2 module, and
- A main program module.

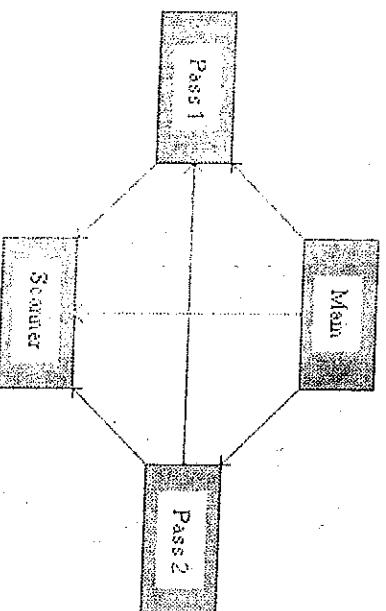
Lexical Analysis: The Scanner: The primary purpose of the scanner module is processing characters into higher level units that are more meaningful for the pass 1 and pass 2 modules. These units are called *tokens*. The process of forming these groups is called *lexical analysis*. The need for a scanner arises in most programs that deal with complex input.

Building the Symbol Table: Pass 1: During pass 1, the input is read and memory addresses are assigned to program labels. Memory is allocated sequentially so that the pass 1 module can use a location counter. For each input statement, this counter is incremented by the size of memory allocated. Whenever a label is encountered, it is recorded in the symbol table. The address assigned is the value of the location counter at the beginning of the statement.

Generating Output: Pass 2: The primary effort in pass 2 is translating instructions into machine code. If the assembler is mixing data and instructions in the same area of memory then translation of data must be done in pass 2. For assemblers that use separate areas of memory for data and instructions, translation of data could be moved into pass 1.

The Main Program: The main program in an assembler is not complex. The primary work is providing file parameters for function calls to the other modules and passing an error Boolean from pass 1 to pass 2 so that an executable output is not generated when there is an error in the assembly language source.

Communication between Modules



The communication between the main program and the two pass modules is quite simple - the main program just calls pass 1 or 2 functions directing them to do their work in the proper order. The functions do not return any data except for possibly an error indication.

The communication between the main program and the scanner is also simple. The name of the file to be assembled is known directly in the main program. The main program either passes the name to the scanner or opens the file and passes it to the scanner. This could be done indirectly through the pass 1 and pass 2 modules.

The communication between pass 1 and pass 2 involves symbol table information. Pass 2 needs to get addresses for labels and values of defined constants from the symbol table. Although there is a fair amount of communication, the interface is simple. It can be a standard table interface.

The scanner is the communication focal point of an assembler. All of the other modules communicate with the scanner. The communication between the pass modules and the scanner is more complex than the communication along other pathways. For this reason, the best place to start working on assembler communication is the scanner client interface. This is an important aspect of assembler design. It cannot be taken lightly.

Advantages and Disadvantages of Assembler

Assembly language is a low-level programming language that is specific to a particular computer architecture, such as x86 or ARM. It is a symbolic representation of machine code, with each assembly language instruction representing a specific machine code operation. The assembler translates these

instructions into the corresponding machine code and outputs it in a form that can be loaded into memory and executed by the computer.

Advantages of Assembler

Assemblers are programs that allow developers to write code in assembly language, which is a low-level programming language that is closer to the machine code that computers execute. Here are some advantages of using an assembler:

- **Speed:** Programs written in assembly language can be faster than those written in higher-level languages because they are closer to the machine code that the computer executes and do not require the additional translation steps that higher-level languages do.
- **Control:** Assembler programs allow developers to have greater control over the hardware and the memory layout of their programs, which can be useful for optimizing performance and accessing low-level features of the system.
- **Size:** Assembler programs can be smaller in size than programs written in higher-level languages because they do not require the use of libraries and other support codes.
- **Portability:** Assembler programs can be more portable than programs written in higher-level languages because they do not depend on specific libraries or runtime environments.
- **Learning:** Assemblers can be a useful tool for learning about computer architecture and how computers work at a low level.
- **Accessibility:** It is more accessible than machine language.

Disadvantages of Assembler

- It is difficult to write and debug programs in assemblers, as there is no high-level language syntax to help catch errors.
- Assembler programs are often more difficult to read and understand than programs written in a high-level language.
- Assembler programs are generally slower and less efficient than programs written in a high-level language and compiled into machine code.
- Assembler programs are machine-dependent, meaning they can only be run on a specific type of computer. This can make it difficult to port programs from one machine to another.
- Assembler programs are more prone to errors and bugs than programs written in a high-level language, as they require the programmer to manually handle low-level details of the computer's architecture.

While assembler can be useful for certain specialized tasks, it is generally not the best choice for most programming tasks due to its complexity and inefficiency.

Why do We Use Assemblers?

- Assemblers allow us to write programs in a language that is closer to the machine language that the computer understands. This makes it easier to write low-level code, such as operating system kernels and device drivers.
- Assemblers allow us to write code that is more efficient than code written in higher-level languages. This is because the code produced by an assembler is typically more compact and executes faster than equivalent code written in a higher-level language.
- Assemblers allow us to write code that has direct control over the hardware, which is not possible with higher-level languages.
- Assemblers are useful for learning about computer architecture and how a computer works at a low level.
- Assemblers are useful for debugging and troubleshooting problems with a system, as they provide a direct connection to the hardware and can help identify issues that may be difficult to detect using higher-level languages.

Use of Assembler

Assemblers are used to create executables that can be run on a computer. They are commonly used to write operating system kernels, device drivers, and other low-level code that requires direct control over the hardware.

Assemblers are also used to optimize code for performance. Because assembly language code is closer to the machine language that the computer understands, it can be more efficient than code written in higher-level languages.

Assemblers are an important tool for low-level programming and for understanding how computers work at a fundamental level.

Applications of Assembler

Some common applications of assembler include:

- **Operating system development:** Assembler is often used to write the core components of an operating system, such as the boot loader and kernel.
- **Device drivers:** Assembler is often used to write device drivers, which are programs that allow the operating system to communicate with hardware devices such as printers, keyboards, and graphics cards.
- **Low-level system programming:** Assembler is often used to write programs that need to interact directly with the hardware of a computer, such as programs that control the memory or the processor.
- **Embedded systems:** Assembler is often used to write programs for embedded systems, which are small, specialized computers that are used to control other devices or systems.
- **Performance-critical applications:** Assembler is often used to write programs that need to be very fast or efficient, as it allows a programmer to have fine-grained control over the instructions that are executed by the computer.

INTERPRETATION, TRANSLATION AND COMPILATION

Compilers, interpreters, translate programs written in high-level languages into machine code that a computer understands. And assemblers translate programs written in low-level or assembly language into machine code. In the compilation process, there are several stages. To help programmers write error-free code, tools are available.

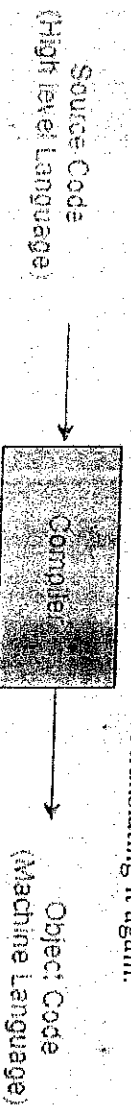
Assembly language is machine-dependent, yet mnemonics used to represent instructions in it are not directly understandable by machine and high-level language is machine-independent. A computer understands instructions in machine code, i.e. in the form of 0s and 1s. It is a tedious task to write a computer program directly in machine code. The programs are written mostly in high-level languages like Java, C++, and Python etc. and are called source code. These source code cannot be executed directly by the computer and must be converted into machine language to be executed. Hence, a special translator system software is used to translate the program written in a high-level language into machine code is called Language Processor and the program after translated into machine code (object program/object code).

The language processors can be any of the following three types:

Compiler:

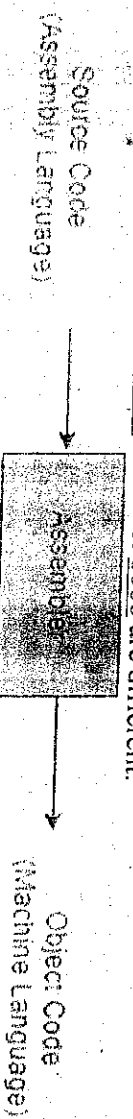
The language processor that reads the complete source program written in high-level language as a whole in one go and translates it into an equivalent program in machine language is called a Compiler. Example: C, C++, C#, Java.

In a compiler, the source code is translated to object code successfully if it is free of errors. The compiler specifies the errors at the end of the compilation with line numbers when there are any errors in the source code. The errors must be removed before the compiler can successfully recompile the source code again the object program can be executed number of times without translating it again.



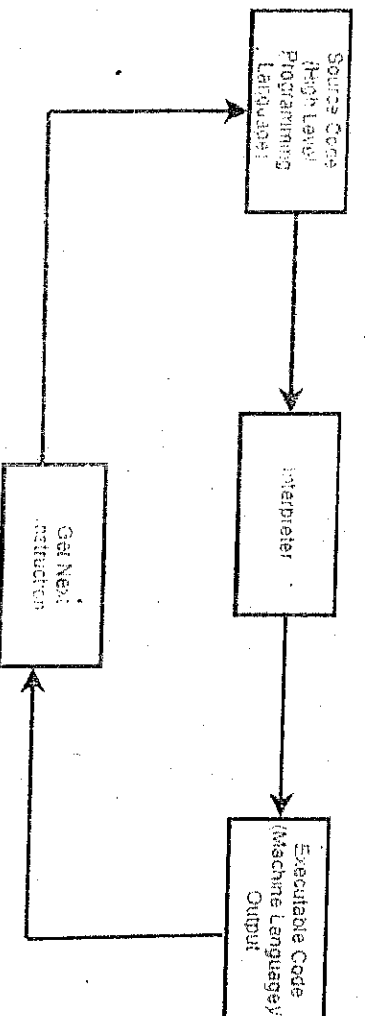
Assembler:

The Assembler is used to translate the program written in Assembly language into machine code. The source program is an input of an assembler that contains assembly language instructions. The output generated by the assembler is the object code or machine code understandable by the computer. Assembler is basically the 1st interface that is able to communicate humans with the machine. We need an Assembler to fill the gap between human and machine so that they can communicate with each other. Code written in assembly language is some sort of mnemonics (instructions) like ADD, MUL, MUX, SUB, DIV, MOV and so on. and the assembler is basically able to convert these mnemonics in Binary code. Here, these mnemonics also depend upon the architecture of the machine. For example, the architecture of Intel 8085 and Intel 8086 are different.



Interpreter:

The translation of a single statement of the source program into machine code is done by a language processor and executes immediately before moving on to the next line is called an interpreter. If there is an error in the statement, the interpreter terminates its translating process at that statement and displays an error message. The interpreter moves on to the next line for execution only after the removal of the error. An Interpreter directly executes instructions written in a programming or scripting language without previously converting them to an object code or machine code. An interpreter translates one line at a time and then executes it. Example: Perl, Python and Matlab.



Difference between Compiler and Interpreter

Compiler	Interpreter
A compiler is a program that converts the entire source code of a programming language into executable machine code for a CPU.	An interpreter takes a source program and runs it line by line, translating each line as it comes to it
The compiler takes a large amount of time to analyze the entire source code but the overall execution time of the program is comparatively faster.	An interpreter takes less amount of time to analyze the source code but the overall execution time of the program is slower.
The compiler generates the error message only after scanning the whole program, so debugging is comparatively hard as the error can be present anywhere in the program.	Its Debugging is easier as it continues translating the program until the error is met.
The compiler requires a lot of memory for generating object codes.	It requires less memory than a compiler because no object code is generated.
Generates intermediate object code.	No intermediate object code is generated.
For Security purpose compiler is more useful.	The interpreter is a little vulnerable in case of security.
Examples: C, C++, Java	Examples: Python, Perl, JavaScript, Ruby

Difference between Assembler and Interpreter:

Assembler	Interpreter
It converts low-level language to the machine language.	It converts high-level language to the machine language.
The program for an Assembler is written for particular hardware.	The program for an Interpreter is written for particular language.
It is one to one i.e. one instruction translates to only one instruction.	It is one to many i.e. one instruction translates to many instruction.
It translates entire program before running.	It translates program instructions line by line.
Errors are displayed before program is running.	Errors are displayed for the every interpreted instruction (if any).
It is used only one time to create an executable file.	It is used every time when the program is running.
Requirement of memory is less.	Requirement of memory is more.
Programming language that it convert is Assembly language.	Programming language that it convert are PHP, Python, Perl, Ruby.

Difference between Compiler and Assembler:

Compiler	Assembler
Compiler converts the source code written by the programmer to a machine level language.	Assembler converts the assembly code into the machine code.

Compiler input <u>source code</u> .	Assembler input <u>assembly language code</u> .
It converts the whole code into <u>machine language</u> at a time.	But the Assembler can't do this at once.
It takes less execution time compared to an assembler.	It takes more time than a compiler.
It shows the whole program error after the whole program is scanned.	It detects errors in the first phase, fixes them, and then the second phase is start to execute.
A Compiler is more intelligent than an Assembler.	But, an Assembler is less intelligent than a Compiler.
The compilation phases are <u>lexical analyzer</u> , <u>syntax analyzer</u> , <u>semantic analyzer</u> , <u>intermediate code generated</u> , <u>a code optimizer</u> , <u>code generator</u> , and <u>error handler</u>	Assembler makes two phases over the given input, first phase and the second phase.
The output of compiler is a mnemonic version of machine code.	The output of assembler is binary code.
<u>C</u> , <u>C++</u> , <u>Java</u> , and <u>C#</u> are examples of compiled languages.	<u>GAS</u> , <u>GNU</u> is an example of an assembler.